JAVA EXCEPTION HANDLING

Core Java Study Notes | Theory, Practical Examples & Interview Revision

Prepared by Rahul Kirpekar

Prepared for classroom learning and student revision

1. What is an Exception?

An exception is an event that occurs during program execution and interrupts the normal flow of a program. Examples include dividing an integer by zero, accessing an invalid array index, or converting an invalid string into a number.

Java provides exception-handling mechanisms so that a program can respond to such situations in a controlled way. Exception handling does not automatically fix the underlying problem; it defines how the application responds to it.

2. Why Do We Need Exception Handling?

- Handle errors in a controlled manner instead of allowing an abrupt failure where recovery is possible.
- Display meaningful messages to users.
- Keep normal business logic separate from error-handling logic.
- Make applications easier to debug and maintain.
- Allow execution to continue after a handled exception, from the appropriate point in the program.

3. Common Runtime Exceptions

ArithmeticException

Occurs during an invalid arithmetic operation, such as integer division by zero.

```
int a = 10;
int b = 0;
System.out.println(a / b);
```

ArrayIndexOutOfBoundsException

Occurs when an array is accessed using an index outside its valid range.

```
int a[] = new int[5];
a[5] = 100;  // Valid indexes are 0 to 4
```

NullPointerException

Occurs when code uses a null reference in a way that requires an actual object.

```
String name = null;
System.out.println(name.length());
```

NumberFormatException

Occurs when a string cannot be converted to the requested numeric type.

```
String value = "a1a0";
int no = Integer.parseInt(value);
```

StringIndexOutOfBoundsException

Occurs when a string is accessed using an invalid character index.

```
String value = "royal";
System.out.println(value.charAt(5));  // Valid indexes: 0 to 4
```

InputMismatchException

Occurs when a Scanner method expects one kind of input but receives an incompatible token.

```
Scanner sc = new Scanner(System.in);
int no = sc.nextInt();  // Entering hello causes a mismatch
```

ClassCastException

Occurs when an object is cast to an incompatible type at runtime.

```
A objA = new B();
C objC = (C) objA;  // Fails if B and C are separate subclasses of A
```

Note: All exceptions listed above are unchecked exceptions. InputMismatchException is in java.util; the others shown are also runtime exceptions.

4. What Happens When an Exception Occurs?

When Java encounters an exception, the statement that caused it fails. Unless the exception is handled, the exception propagates up the call stack. Statements later in the same execution path are skipped.

```
int no1 = 10;
int no2 = 0;
int ans = no1 / no2;
System.out.println("Division : " + ans);
```

The division throws ArithmeticException, so the Division print statement is not executed.

5. Exception Handling Using try-catch

The try block contains code that may throw an exception. A matching catch block handles that exception.

```
try {
    // Code that may cause an exception
}
catch (ExceptionType e) {
    // Code to handle the exception
}
```

Practical example: Division

```
int no1 = 10;
int no2 = 0;
int ans = 0;

try {
    ans = no1 / no2;
}
catch (ArithmeticException e) {
    System.out.println(e.getMessage());
    System.out.println("Exception handled");
```

```
}

System.out.println("Ans : " + ans);
```

Explanation:

1. The try block attempts the division.
2. Division by zero throws ArithmeticException.
3. The matching catch block handles the exception.
4. The assignment fails, so ans remains 0.
5. Execution continues after the try-catch structure.

Important: Java does not resume execution from the failed statement inside the try block. It moves to the matching catch block and then continues after the try-catch structure.

6. Useful Exception Object Methods

Method	Purpose
e.getMessage()	Returns the detail message, if available.
e.printStackTrace()	Prints exception details and stack frames to help locate the problem.

```
catch (ArithmeticException e) {
    System.out.println(e.getMessage());
    e.printStackTrace();
}
```

Industry practice: Use stack traces for debugging and logging. Show users a clear message rather than exposing internal implementation details.

7. Multiple catch Blocks

A single try block can be followed by multiple catch blocks. Each catch block handles a different exception type.

```
try {
    ans = no1 / no2;
    a[5] = 100;
    String name = null;
    System.out.println(name.length());
}
catch (ArithmeticException e) {
    e.printStackTrace();
}
catch (ArrayIndexOutOfBoundsException e) {
    e.printStackTrace();
}
catch (NullPointerException e) {
    e.printStackTrace();
}
catch (Exception e) {
    e.printStackTrace();
}
```

If the inputs are 10 and 2, division succeeds. The statement a[5] = 100 then throws ArrayIndexOutOfBoundsException. The matching catch block handles it, and name.length() is never reached in that execution.

Rule: Place specific catch blocks before general catch blocks. If catch (Exception e) appears first, later catches for its subclasses become unreachable and the code will not compile.

8. Multi-catch in Java

Multi-catch, introduced in Java 7, lets one catch block handle multiple exception types when the handling logic is the same.

```
try {
    ans = no1 / no2;
    a[5] = 100;
    String name = null;
    System.out.println(name.length());
}
catch (ArithmeticException | ArrayIndexOutOfBoundsException |
       NullPointerException e) {
    e.printStackTrace();
}
catch (Exception e) {
    e.printStackTrace();
}
```

- Reduces duplicate code.
- Keeps common handling logic in one place.
- Exception types in a multi-catch cannot have a superclass-subclass relationship with each other.
- For example, Exception | ArithmeticException is invalid because ArithmeticException is already covered by Exception.

9. Java Exception Hierarchy

```
Object
  └── Throwable
       ├── Error
       └── Exception
            ├── RuntimeException
            │    ├── ArithmeticException
            │    ├── NullPointerException
            │    ├── ClassCastException
            │    ├── NumberFormatException
            │    ├── ArrayIndexOutOfBoundsException
            │    └── StringIndexOutOfBoundsException
            └── Other checked exceptions, e.g. IOException
```

InputMismatchException is also an unchecked exception; it is in the java.util package and extends NoSuchElementException, which is part of the RuntimeException hierarchy.

Checked vs. Unchecked Exceptions

Point	Checked exceptions	Unchecked exceptions

Compiler requirement	Must generally be caught or declared with throws.	No compile-time catch-or-declare requirement.
Example	IOException	ArithmeticException
Handling	Can be caught or declared.	Can also be caught; handling is optional at compile time.

10. How to Avoid Common Exceptions

Exception	Recommended practice
ArithmeticException	Validate the divisor before integer division.
ArrayIndexOutOfBoundsException	Check that the index is between 0 and array.length - 1.
NullPointerException	Validate references before using them; avoid unnecessary null values.
NumberFormatException	Validate numeric strings before parsing.
StringIndexOutOfBoundsException	Check string length before calling charAt(index).
InputMismatchException	Validate input and handle invalid tokens appropriately.
ClassCastException	Check object types and use polymorphism or safe type checks where appropriate.

11. Interview Revision Questions

6. **Which exception occurs when an integer is divided by zero?**
   Answer: ArithmeticException
7. **What are valid indexes for new int[5]?**
   Answer: 0 to 4
8. **Which method prints an exception and its call stack?**
   Answer: printStackTrace()
9. **Which catch block should normally come first?**
   Answer: The most specific exception
10. **Which Java version introduced multi-catch?**
    Answer: Java 7
11. **What happens after a matching catch block finishes?**
    Answer: Execution continues after the try-catch structure
12. **Which exception can Integer.parseInt("a1a0") throw?**
    Answer: NumberFormatException
13. **What kind of exception is NullPointerException?**
    Answer: Unchecked exception

12. Key Takeaways

14. An exception interrupts the normal flow of execution.
15. try contains code that may throw an exception; catch handles a matching exception.
16. Multiple catch blocks support different handling strategies.
17. Multi-catch handles multiple exception types in one block.
18. getMessage() retrieves the exception message; printStackTrace() helps debugging.
19. Exception is more general than RuntimeException and its subclasses.
20. An uncaught exception typically terminates the affected thread, not necessarily the entire JVM.

21. Combine exception handling with input validation and sound program design.

Trainer's Classroom Activity

Ask students to run each example first without exception handling and observe the error. Then ask them to add a suitable try-catch block. Finally, ask them to explain which statements execute and which are skipped when an exception occurs.

Suggested practice: modify the division example, invalid array index example, null String example, string-to-number conversion example, invalid character index example, Scanner input example, and invalid object-cast example.